

JXPAMG 中基本运算的接口说明

岳孝强 徐小文 舒适

2021 年 2 月 10 日

目 录

1	预备知识	2
1.1	JXPAMG 简介	2
1.2	CSR 存储格式	2
1.3	JXPAMG 中的矩阵结构体	4
1.3.1	串行矩阵	4
1.3.2	串行矩阵的并行分划	4
1.3.3	并行矩阵	5
1.4	JXPAMG 中的向量结构体	7
1.4.1	串行向量	7
1.4.2	并行向量	8
2	JXPAMG 中基本运算的接口	9
2.1	matops	9
2.2	mvops	15
2.3	relaxations	16
2.4	vecops	20

本文档将给出JXPAMG 中基本运算中各个函数的接口说明, 为此首先给出预备知识.

§1 预备知识

§1.1 JXPAMG 简介

JXPAMG (Parallel AMG Solver Developed by JiuSuo (IAPCM) and XTU) 是由北京应用物理与计算数学研究所和湘潭大学联合开发的并行代数多重网格 (AMG) 解法器软件包, 主要针对武器物理、激光聚变、工程力学与电磁环境等重大应用数值模拟中出现的大规模病态离散线性系统的求解而发展的基于 AMG 的快速算法库.

JXPAMG 以经典 AMG 算法 [7, 11] 为基础, 并结合了近年发展起来的面向应用特征、基于部分容易获取的几何与分析信息的 AMG (GAMG) 算法 [1, 2, 5, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23], 如自适应物理量粗化的两水平算法 (APCTL) [18, 23]、多尺度 AMG [12]、高次元 AMG [14]、ILU-AMG 组合型算法 [21, 22]、aSetup-AMG [19, 20] 和基于代数界面粗化的 AMG (AI-AMG) [17] 等.

JXPAMG 采用面向复用的软件设计思想, 软件架构包含算法层与基本运算层, 其中算法层包含了 AMG 算法的基本组件, 包括 Setup 阶段的网格粗化、插值算子构造和粗网格矩阵计算以及迭代求解过程 (Cycle) 的光滑迭代、插值和限制、粗网格求解等, 基本运算层则封装稀疏矩阵和向量的数据结构、以及相关操作的并行实现和面向体系结构的性能优化. 基于层次化架构, JXPAMG 实现了算法设计与具体实现的分离 (详见示意图 2). 目前已经封装了由美国 Lawrence Livermore 国家实验室的应用科学计算中心开发的经典 AMG 并行解法器 BoomerAMG [3], 集成了一系列应用驱动的 GAMG 算法模块以及 Krylov 子空间类通用迭代方法 [8], 并且已经无缝对接编程框架 JASMIN [6] 和 JAUMIN [4].

§1.2 CSR 存储格式

CSR (Compressed Sparse Row) 格式是存储稀疏矩阵的最常用格式, 该格式需用到以下三个一维数组:

1. $AA(i)$, $i = 1(1)nz$: a real array, contains the real value a_{ij} stored row by row, from row 1 to row n . (实型数组, 按行存放矩阵 A 中的所有非零元素.)
2. $JA(i)$, $i = 1(1)nz$: an integer array, contains the column indices of the elements a_{ij} as stored in the array AA . (整型数组, 依次存放数组 AA 中对应元素的列号.)
3. $IA(i)$, $i = 1(1)n + 1$: an integer array, contains the pointers to the beginning of each row in the arrays AA and JA . Thus, the content of $IA(i)$, $i = 1(1)n$ is the position in arrays AA and JA where the i -th row starts, and $IA(n + 1) = IA(1) + nz$ can be taken as the address in AA and JA of the beginning of a fictitious row number $n + 1$. (整型数组, 依次存放各行在数组 AA 或 JA 中存储的首地址. 当 j 从 $IA(i)$ 到 $IA(i + 1) - 1$ 变化时, $AA(j)$ 即可遍历矩阵 A 第 i 行的所有非零元素.)

JXPAMG: Parallel AMG Solver Developed by JiuSuo (IAPCM) and XTU

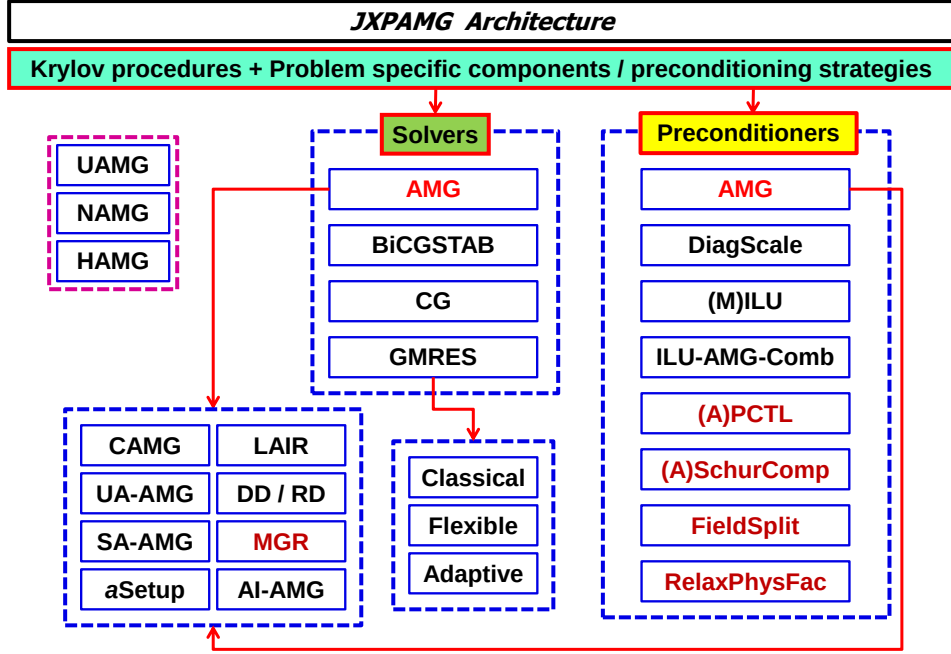


图 1 JXPAMG 框架图

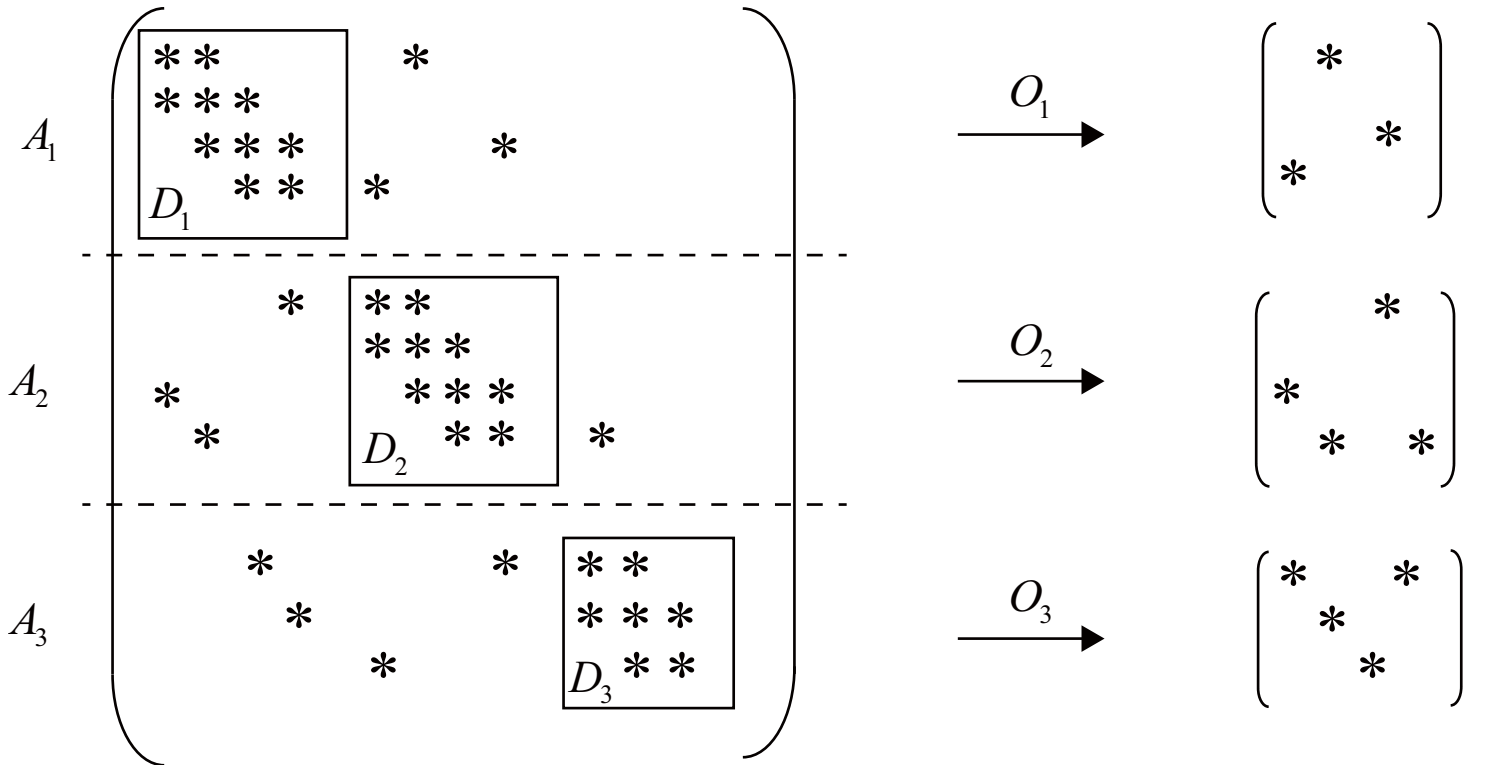


图 2 JXPAMG 框架图

注 1.1 数组 AA 中按行存放矩阵 A 的非零元素时, 每一行非零元素的顺序可以是任意的, 通常按照按列号从小到大的顺序来存放, 有时为方便访问对角线元素, 在每一行也常按照先对角线元素后非对角线元素的顺序来存放.

注 1.2 若矩阵 A 的第 i 行没有非零元素, 则有 $IA(i) = IA(i + 1)$.

例 1.1 矩阵

$$A = \begin{pmatrix} 1. & 0. & 2. & 0. \\ 0. & 3. & 0. & 4. \\ 5. & 0. & 6. & 7. \\ 8. & 0. & 0. & 9. \end{pmatrix}$$

的 CSR 存储格式为

AA	1.	2.	4.	3.	6.	5.	7.	8.	9.
JA	1	3	4	2	3	1	4	1	4
IA	1	3	5	8	10				

表 1 矩阵 A 的 CSR 存储格式

§1.3 JXPAMG 中的矩阵结构体

§1.3.1 串行矩阵

串行 CSR 矩阵的结构体 `jx_CSRMatrix` 为

```
typedef struct
{
    double *data; // 指向非零元数组存储空间首地址的指针
    int *i; // 指向行索引数组存储空间首地址的指针
    int *j; // 指向列指标数组存储空间首地址的指针
    int num_rows; // 行数
    int num_cols; // 列数
    int num_nonzeros; // 非零元个数
    int *rownnz; // 指向非零行(行非零元个数不为 0)存储空间首地址的指针
    int num_rownnz; // 非零行的数量
    int owns_data;
    // 标记值. 1: 表示该串行矩阵有自己的 data 数组和 j 数组, 0: 表示没有
} jx_CSRMatrix;
```

§1.3.2 串行矩阵的并行分划

以 3 个进程为例, 图 3 给出一个 8×8 矩阵“均分”到所有进程的并行分划示意图.

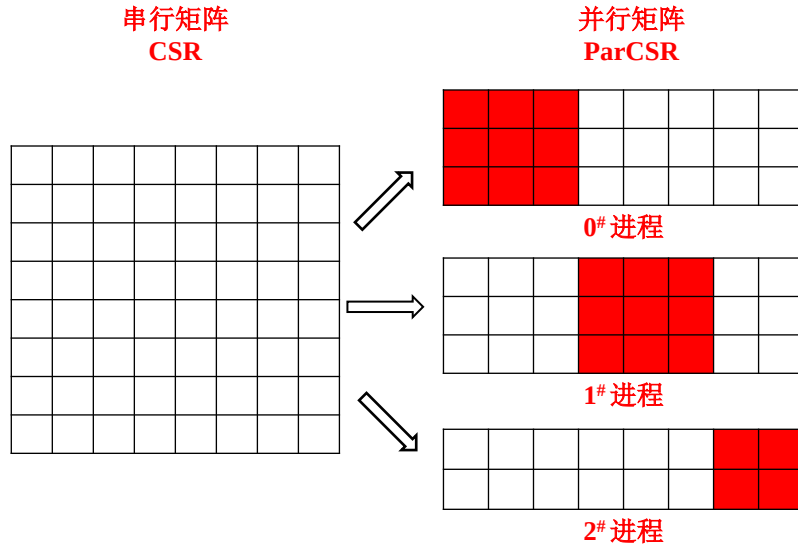


图 3 串行矩阵的并行分划示意图

§1.3.3 并行矩阵

并行 CSR 矩阵的结构体 `jx_ParCSRMatrix` 为

```
typedef struct
{
    MPI_Comm          comm;           //通信域
    int               global_num_rows; //整个矩阵的行数
    int               global_num_cols; //整个矩阵的列数
    int               first_row_index; //并行矩阵中第一行对应于整体矩阵的行号
    int               first_col_diag;  //并行矩阵对角块第一列对应于整体矩阵的列号
    int               last_row_index;  //并行矩阵中最后一行对应于整体矩阵的行号
    int               last_col_diag;   //并行矩阵对角块最后一列对应于整体矩阵的列号
    jx_CSRMatrix      *diag;           //指向并行矩阵的对角块存储空间的首地址的指针
    jx_CSRMatrix      *offd;           //指向并行矩阵的非对角块存储空间的首地址的指针
    int               *col_map_offd;   //指向非对角块各非零元列编号到整体矩阵列编号的
                                      //映射数组的存储空间首地址的指针
    int               *row_starts;     //指向行分划数组存储空间首地址的指针
    int               *col_starts;     //指向列分划数组存储空间首地址的指针
    jx_ParCSRCommPkg *comm_pkg;        //指向矩阵 A 的通信包的首地址的指针
    jx_ParCSRCommPkg *comm_pkgT;       //指向矩阵 A^T 的通信包的首地址的指针
    int               owns_data;
    //标记值, 1: 表示该并行矩阵有自己的对角块和非对角块, 0: 表示没有
    int               owns_row_starts;
    //标记值, 1: 表示该并行矩阵有自己的行分划, 0: 表示没有
}
```

```

int                owns_col_starts;
//标记值, 1: 表示该并行矩阵有自己的列分划, 0: 表示没有
// 以上三个标记值将影响最终并行矩阵的释放, 因此这里需要特别注意,
// 若不存在自己对应的成员, 如:行分划数组共用别的并行向量或矩阵的指针,
// 且两者标记值均为 1, 则会出现重复释放的问题.
int                num_nonzeros;    // 整个矩阵的非零元个数
// 以下几个为不常用的成员变量 (在指定分划数组时使用)
double            d_num_nonzeros;
// 整个矩阵的非零元个数, 但注意这里的变量类型为实型.
int                *rowindices;
double            *rowvalues;
int                getrowactive;
jx_IJAssumedPart *assumed_partition; /* only populated if no_global_partition
                                         option is used (compile-time option) */
} jx_ParCSRMatrix;

```

注 1.3 *col_map_offd* 表示非对角块各非零列 (有非零元的列) 的整体编号, 此时

1. 数组长度: 非对角块非零列的个数;
2. *offd_j*: 存放的是非对角块各非零元对应的局部列号. (同样也可理解为非对角块各非零元对应的整体列号在 *col_map_offd* 数组中的位置.) 注意这时, 非对角块指的是并行矩阵中除去对角块及非零列后得到的矩阵.

例 1.2 矩阵

$$A = \begin{pmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{pmatrix}.$$

当进程数为 2 时, 每个进程对应的并行矩阵 *jx_ParCSRMatrix* 中几个常用成员的具体取值.

表 2 两个进程对应并行矩阵的成员

结构体成员		my_id=0	my_id=1
global_num_rows		4	4
global_num_cols		4	4
first_row_index		0	2
first_col_diag		0	2
last_row_index		1	3
last_col_diag		1	3
diag	data	{4,-1,-1,4}	{4,-1,-1,4}
	i	{0,2,4}	{0,2,4}
	j	{0,1,0,1}	{0,1,0,1}
	num_rows	2	2
	num_cols	2	2
	num_nonzeros	4	4
offd	data	{-1,-1}	{-1,-1}
	i	{0,1,2}	{0,1,2}
	j	{0,1}	{0,1}
	num_rows	2	2
	num_cols	2	2
	num_nonzeros	2	2
col_map_offd		{2,3}	{0,1}
row_starts		{0,2,4}	{0,2,4}
col_starts		{0,2,4}	{0,2,4}
num_nonzeros		12	12

§1.4 JXPAMG 中的向量结构体

§1.4.1 串行向量

串行向量的结构体 jx_Vector 为

```
typedef struct
{
    double  *data;          // 指向元素值数组存储空间首地址的指针
    int      size;          // 向量的长度
    int      owns_data;
    //标记值, 1: 表示该向量有自己的 data, 0: 表示没有
    int      num_vectors;    // 该向量存放的子向量个数
    int      multivec_storage_method; // 多个子向量存放的方法.
```

```

// 存放多个向量时，这里每个子向量的长度为 size，且所有元素均存放在 data 数组中
int      vecstride; // 表示每个子向量中相同编号的元素在 data 中位置之间的偏移量
int      idxstride; // 表示各个子向量中相邻编号元素在 data 中位置之间的偏移量
} jx_Vector;

```

§1.4.2 并行向量

并行向量的结构体 `jx_ParVector` 为

```

typedef struct
{
    MPI_Comm      comm;          // 通信域
    int           global_size;   // 整体向量的长度
    int           first_index;   // 当前进程中向量的第一个元素在整体向量中的编号
    int           last_index;    // 当前进程中向量的最后一个元素在整体向量中的编号
    int           *partitioning; // 指向进程分划数组存储空间首地址的指针
    jx_Vector     *local_vector; // 指向当前进程所对应的子向量存储空间首地址的指针
    int           owns_data;
    // 标记值，1: 表示该并行向量有自己的 local_vector, 0: 表示没有
    int           owns_partitioning;
    // 标记值，1: 表示该并行向量有自己的 partitioning, 0: 表示没有
    jx_IJAssumedPart *assumed_partition;
    /* only populated if no_global_partition option
       is used (compile-time option) AND this partition
       needed(for setting off-proc elements, for example) */
} jx_ParVector;

```

例 1.3 向量 $x = (1, 2, 3, 4, 5, 6)^T$. 当进程数为 2 时，每个进程对应的并行向量 `jx_ParVector` 中几个常用成员的具体取值.

表 3 两个进程对应并行向量的成员

结构体成员		my_id=0	my_id=1
global_size		6	6
first_index		0	3
last_index		2	5
partitioning		{0,3,6}	{0,3,6}
local_vector	data	{1,2,3}	{4,5,6}
	size	3	3
	owns_data	1	1
	num_vectors	1	1
	multivec_storage_method	0	0
	vecstride	3	3
	idxstride	1	1

§2 JXPAMG 中基本运算的接口

§2.1 matops

该文件夹下有 6 个 C 文件, 其函数名及接口分别为

1. par_csr_matop.c

- 并行矩阵的复制

(a) 入口: 并行矩阵 A、B, 标志值 copy_data;

(b) 出口: 并行矩阵 B;

(c) 功能: 若 copy_data = 0, 则仅将 A 的非零结构复制给 B; 若 copy_data = 1, 还有元素值.

```
int
jx_ParCSRMatrixCopy( jx_ParCSRMatrix *A,
                     jx_ParCSRMatrix *B,
                     int                copy_data )
```

- 并行矩阵的转置

(a) 入口: 并行矩阵 A, 标志值 data;

(b) 出口: 并行矩阵 *AT_ptr;

(c) 功能: 若 data = 0, 则仅将 A 的非零结构作转置到 *AT_ptr 中; 若 data = 1, 还有元素值.

```

int
jx_ParCSRMatrixTranspose( jx_ParCSRMatrix *A,
                           jx_ParCSRMatrix **AT_ptr,
                           int                data )

```

- 并行矩阵的提取

- (a) 入口: 并行矩阵 B、A, 标志值 data;
- (b) 出口: 串行矩阵;
- (c) 功能: 若 data = 0, 则将落在其它进程上的并行矩阵 B 中与当前进程上并行矩阵 A 进行“局部”相乘所必需的所有“行”的非零结构; 若 data = 1, 还有元素值.

```

jx_CSRMatrix *
jx_ParCSRMatrixExtractBExt( jx_ParCSRMatrix *B,
                             jx_ParCSRMatrix *A,
                             int                data )

```

- 并行矩阵提取的子函数:

```

void
jx_ParCSRMatrixExtractBExt_Arrays( int                **pB_ext_i,
                                    int                **pB_ext_j,
                                    double              **pB_ext_data,
                                    int                **pB_ext_row_map,
                                    int                *num_nonzeros,
                                    int                data,
                                    int                find_row_map,
                                    MPI_Comm           comm,
                                    jx_ParCSRCommPkg   *comm_pkg,
                                    int                num_cols_B,
                                    int                num_recvs,
                                    int                num_sends,
                                    int                first_col_diag,
                                    int                first_row_index,
                                    int                *recv_vec_starts,
                                    int                *send_map_starts,
                                    int                *send_map_elmts,
                                    int                *diag_i,
                                    int                *diag_j,

```

```

int          *offd_i,
int          *offd_j,
int          *col_map_offd,
double       *diag_data,
double       *offd_data )

```

- 并行矩阵乘积的子函数:

```

void
jx_ParMatmul_RowSizes( int **C_diag_i,
                      int **C_offd_i,
                      int **B_marker,
                      int  *A_diag_i,
                      int  *A_diag_j,
                      int  *A_offd_i,
                      int  *A_offd_j,
                      int  *B_diag_i,
                      int  *B_diag_j,
                      int  *B_offd_i,
                      int  *B_offd_j,
                      int  *B_ext_diag_i,
                      int  *B_ext_diag_j,
                      int  *B_ext_offd_i,
                      int  *B_ext_offd_j,
                      int  *map_B_to_C,
                      int  *C_diag_size,
                      int  *C_offd_size,
                      int   num_rows_diag_A,
                      int   num_cols_offd_A,
                      int   allsquare,
                      int   num_cols_diag_B,
                      int   num_cols_offd_B,
                      int   num_cols_offd_C )

```

- 并行矩阵的乘积

- (a) 入口: 并行矩阵 A、B;
- (b) 出口: 并行矩阵 $C = A \times B$, 但 $A \times B$ 不含 row_starts 和 col_starts.

```

jx_ParCSRMatrix *
jx_ParMatmul( jx_ParCSRMatrix  *A, jx_ParCSRMatrix *B )

```

- 并行矩阵的乘积

(a) 入口: 并行矩阵 A、B;

(b) 出口: 并行矩阵 $C = A \times B$, 但 $A \times B$ 含有 row_starts 和 col_starts.

```
jx_ParCSRMatrix *
jx_ParMatmul_OwnPartition( jx_ParCSRMatrix *A, jx_ParCSRMatrix *B )
```

2. par_gs.c

- Gauss 消去法

(a) 入口: 系数矩阵 A (长度 size×size, 满矩阵), 右端向量 x, 矩阵阶数 size;

(b) 出口: 解向量 x;

```
int
jx_gselim( double *A_matrix, double *x_vector, int size )
```

3. par_gs_piv.c

- Gauss 主元素消去法

(a) 入口: 系数矩阵 A (长度 size×size, 满矩阵), 右端向量 x, 矩阵阶数 size;

(b) 出口: 解向量 x;

```
int
jx_gselim_piv( double *A_matrix, double *x_vector, int size )
```

4. par_rap.c

- 三并行矩阵相乘的子函数

```
jx_CSRMatrix *
jx_ExchangeRAPData( jx_CSRMatrix *RAP_int, jx_ParCSRCommPkg *comm_pkg_RT )
```

- 三并行矩阵相乘

(a) 入口: 并行矩阵 RT、par_A、par_P;

(b) 出口: 并行矩阵 *RAP_ptr;

(c) 功能: $*RAP_ptr = \text{trans}(RT) \times \text{par_A} \times \text{par_P}$. 常将 RT 取为 par_P.

```
int
jx_PAMGBuildCoarseOperator( jx_ParCSRMatrix *RT,
                             jx_ParCSRMatrix *par_A,
                             jx_ParCSRMatrix *par_P,
                             jx_ParCSRMatrix **RAP_ptr )
```

5. par_rap_omp.c

- OpenMP 下的三并行矩阵相乘

- (a) 入口: 并行矩阵 RT、par_A、par_P;
- (b) 出口: 并行矩阵 *RAP_ptr;
- (c) 功能: $*RAP_ptr = \text{trans}(RT) \times \text{par_A} \times \text{par_P}$. 常将 RT 取为 par_P.

```
int
jx_PAMGBuildCoarseOperatorOMP( jx_ParCSRMatrix *RT,
                                jx_ParCSRMatrix *par_A,
                                jx_ParCSRMatrix *par_P,
                                jx_ParCSRMatrix **RAP_ptr,
                                int                *icor_yoo )
```

6. seq_csr_matop.c

- 串行矩阵的复制

- (a) 入口: 串行矩阵 A、B, 标志值 copy_data;
- (b) 出口: 串行矩阵 B;
- (c) 功能: 若 copy_data = 0, 则仅将 A 的非零结构复制给 B; 若 copy_data = 1, 还有元素值.

```
int
jx_CSRMatrixCopy( jx_CSRMatrix *A, jx_CSRMatrix *B, int copy_data )
```

- 串行矩阵的重排

- (a) 入口: 串行矩阵 A;
- (b) 出口: 串行矩阵 A;
- (c) 功能: 串行矩阵的每行均优先存储主对角元素.

```
int
jx_CSRMatrixReorder( jx_CSRMatrix *A )
```

- 串行矩阵的转置

- (a) 入口: 串行矩阵 A, 标志值 data;
- (b) 出口: 串行矩阵 *AT;
- (c) 功能: 若 data = 0, 则仅将 A 的非零结构作转置到 *AT 中; 若 data = 1, 还有元素值.

```
int
jx_CSRMatrixTranspose( jx_CSRMatrix *A, jx_CSRMatrix **AT, int data )
```

- 串行矩阵的重排

(a) 入口: 串行矩阵 A;

(b) 出口: 串行矩阵 A;

(c) 功能: 串行矩阵中每行均优先存储主对角元素, 且每行的其余元素均是按列号由小到大存储.

```
void  
jx_CSRMatrixReorderColumnNumber12( jx_CSRMatrix *A )
```

- 串行矩阵的重排

(a) 入口: 串行矩阵 A;

(b) 出口: 串行矩阵 A;

(c) 功能: 串行矩阵中每行均优先存储主对角元素, 且每行的其余元素均是按列号由大到小存储.

```
void  
jx_CSRMatrixReorderColumnNumber21( jx_CSRMatrix *A )
```

- 串行矩阵的重排

(a) 入口: 串行矩阵 A;

(b) 出口: 串行矩阵 A;

(c) 功能: 串行矩阵中每行的元素均是按列号由小到大存储.

```
void  
jx_CSRMatrixReorderColumnNumberAll( jx_CSRMatrix *A )
```

- 串行矩阵的相加

(a) 入口: 串行矩阵 A, B;

(b) 出口: 串行矩阵 $C=A+B$.

```
jx_CSRMatrix *  
jx_CSRMatrixAdd( jx_CSRMatrix *A, jx_CSRMatrix *B )
```

- 串行矩阵的尺度化

(a) 入口: 串行矩阵 A, 实数 alpha;

(b) 出口: 串行矩阵 $B=\alpha \times A$.

```
void  
jx_CSRMatrixScale( jx_CSRMatrix *A, double alpha )
```

- 串行矩阵的对称化

(a) 入口: 串行矩阵 A;

(b) 出口: 串行矩阵 $B=[A+\text{trans}(A)]/2$.

```
jx_CSRMatrix *
jx_CSRMatrixSymmetrization( jx_CSRMatrix *A )
```

§2.2 mvops

该文件夹下有 2 个 C 文件, 其函数名及接口分别为

1. par_csr_matvec.c

- 并行矩阵向量乘

(a) 入口: 并行矩阵 A, 并行向量 x、y, 实数 alpha、beta;

(b) 出口: 并行向量 y;

(c) 功能: $y = \alpha \times A \times x + \beta \times y$.

```
int
jx_ParCSRMatrixMatvec( double          alpha,
                       jx_ParCSRMatrix *A,
                       jx_ParVector    *x,
                       double          beta,
                       jx_ParVector    *y  )
```

- 并行矩阵向量乘

(a) 入口: 并行矩阵 A, 并行向量 x、y, 实数 alpha、beta;

(b) 出口: 并行向量 y;

(c) 功能: $y = \alpha \times \text{trans}(A) \times x + \beta \times y$.

```
int
jx_ParCSRMatrixMatvecT( double          alpha,
                       jx_ParCSRMatrix *A,
                       jx_ParVector    *x,
                       double          beta,
                       jx_ParVector    *y  )
```

2. seq_csr_matvec.c

- 串行矩阵向量乘

(a) 入口: 串行矩阵 A, 串行向量 x、y, 实数 alpha、beta;

(b) 出口: 串行向量 y ;

(c) 功能: $y = \alpha \times A \times x + \beta \times y$.

```
int
jx_CSRMatrixMatvec( double      alpha,
                    jx_CSRMatrix *A,
                    jx_Vector    *x,
                    double       beta,
                    jx_Vector    *y    )
```

- 串行矩阵向量乘

(a) 入口: 串行矩阵 A , 串行向量 x 、 y , 实数 α 、 β ;

(b) 出口: 串行向量 y ;

(c) 功能: $y = \alpha \times \text{trans}(A) \times x + \beta \times y$.

```
int
jx_CSRMatrixMatvecT( double      alpha,
                    jx_CSRMatrix *A,
                    jx_Vector    *x,
                    double       beta,
                    jx_Vector    *y    )
```

§2.3 relaxations

该文件夹下有 9 个 C 文件, 其函数名及接口分别为

1. par_relax_0.c

- Jacobi 或 CF-Jacobi 磨光子

(a) 入口: 并行矩阵 par_matrix , 并行向量 par_rhs 、 Vtemp , CF 数组 cf_marker , 磨光点的标志值 relax_points , 磨光系数 relax_weight , 无用系数 ω ;

(b) 出口: 并行向量 par_app .

```
int
jx_PAMGRelax0( jx_ParCSRMatrix *par_matrix,
               jx_ParVector    *par_rhs,
               int              *cf_marker,
               int              relax_points,
               double           relax_weight,
               double           omega,
               jx_ParVector    *par_app,
               jx_ParVector    *Vtemp )
```

2. par_relax_1.c

- Gauss Seidel 或 CF-Gauss Seidel 磨光子: 串行.

- (a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, 无用系数 relax_weight, 无用系数 omega;
- (b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax1( jx_ParCSRMatrix *par_matrix,
                jx_ParVector    *par_rhs,
                int              *cf_marker,
                int              relax_points,
                double           relax_weight,
                double           omega,
                jx_ParVector     *par_app,
                jx_ParVector     *Vtemp )
```

3. par_relax_2.c

- Gauss Seidel 或 CF-Gauss Seidel 磨光子: 内部点并行, 边界点串行.

- (a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, 无用系数 relax_weight, 无用系数 omega;
- (b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax2( jx_ParCSRMatrix *par_matrix,
                jx_ParVector     *par_rhs,
                int              *cf_marker,
                int              relax_points,
                double           relax_weight,
                double           omega,
                jx_ParVector     *par_app,
                jx_ParVector     *Vtemp )
```

4. par_relax_3.c

- 混合或 CF-混合磨光子: 内部点 SOR (forward), 边界点 Jacobi.

- (a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, Jacobi 磨光系数 relax_weight, SOR 磨光系数 omega;

(b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax3( jx_ParCSRMatrix *par_matrix,
                jx_ParVector    *par_rhs,
                int              *cf_marker,
                int              relax_points,
                double            relax_weight,
                double            omega,
                jx_ParVector    *par_app,
                jx_ParVector    *Vtemp )
```

5. par_relax_4.c

- 混合或 CF-混合磨光子: 内部点 SOR (backward), 边界点 Jacobi.

(a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, Jacobi 磨光系数 relax_weight, SOR 磨光系数 omega;

(b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax4( jx_ParCSRMatrix *par_matrix,
                jx_ParVector    *par_rhs,
                int              *cf_marker,
                int              relax_points,
                double            relax_weight,
                double            omega,
                jx_ParVector    *par_app,
                jx_ParVector    *Vtemp )
```

6. par_relax_5.c

- 混合或 CF-混合磨光子: 内部点 Gauss Seidel, 边界点 Jacobi.

(a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, 无用系数 relax_weight, 无用系数 omega;

(b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax5( jx_ParCSRMatrix *par_matrix,
                jx_ParVector    *par_rhs,
```

```

int            *cf_marker,
int            relax_points,
double        relax_weight,
double        omega,
jx_ParVector  *par_app,
jx_ParVector  *Vtemp )

```

7. par_relax_6.c

- 混合或 CF-混合磨光子: 内部点 SSOR, 边界点 Jacobi.

(a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, Jacobi 磨光系数 relax_weight, SOR 磨光系数 omega;

(b) 出口: 并行向量 par_app.

```

int
jx_PAMGRelax6( jx_ParCSRMatrix *par_matrix,
               jx_ParVector    *par_rhs,
               int              *cf_marker,
               int              relax_points,
               double           relax_weight,
               double           omega,
               jx_ParVector    *par_app,
               jx_ParVector    *Vtemp )

```

8. par_relax_7.c

- Jacobi 磨光子: 利用 Matvec 实现, 仅在 CGNR 中调用

(a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、par_app、Vtemp, CF 数组 cf_marker, 磨光点的标志值 relax_points, 磨光系数 relax_weight, 无用系数 omega;

(b) 出口: 并行向量 par_app.

```

int
jx_PAMGRelax7( jx_ParCSRMatrix *par_matrix,
               jx_ParVector    *par_rhs,
               int              *cf_marker,
               int              relax_points,
               double           relax_weight,
               double           omega,
               jx_ParVector    *par_app,
               jx_ParVector    *Vtemp )

```

9. par_relax_9.c

- Gauss 消去法

(a) 入口: 并行矩阵 par_matrix, 并行向量 par_rhs、无用Vtemp, 无用数组 cf_marker, 无用系数 relax_points, 无用系数 relax_weight, 无用系数 omega;

(b) 出口: 并行向量 par_app.

```
int
jx_PAMGRelax9( jx_ParCSRMatrix *par_matrix,
                jx_ParVector    *par_rhs,
                int              *cf_marker,
                int              relax_points,
                double            relax_weight,
                double            omega,
                jx_ParVector      *par_app,
                jx_ParVector      *Vtemp )
```

§2.4 vecops

该文件夹下有 2 个 C 文件, 其函数名及接口分别为

1. par_vecop.c

- 并行向量的复制

(a) 入口: 并行向量 x、y;

(b) 出口: 并行向量 y.

```
int
jx_ParVectorCopy( jx_ParVector *x, jx_ParVector *y )
```

- 并行向量的设定

(a) 入口: 并行向量 v, 实数 value;

(b) 出口: 并行向量 v, 其中 $v[i]=value$.

```
int
jx_ParVectorSetConstantValues( jx_ParVector *v, double value )
```

- 并行向量的范数

(a) 入口: 并行向量 x;

(b) 出口: 并行向量 x 的 1-范数.

```
double
jx_ParVectorNorm1( jx_ParVector *x )
```

- 并行向量的范数

(a) 入口: 并行向量 x 、 y ;

(b) 出口: 并行向量 x 、 y 的逐点 1-范数 $\max\{ |x_i| / |y_i|, i = 0, 1, \dots, n-1 \}$.

```
double
jx_ParVectorPointWiseRelNorm1(jx_ParVector *x, jx_ParVector *y)
```

- 并行向量的随机设定

(a) 入口: 并行向量 v , 种子 $seed$;

(b) 出口: 并行向量 v .

```
int
jx_ParVectorSetRandomValues( jx_ParVector *v, int seed )
```

- 并行向量的尺度化

(a) 入口: 并行向量 y , 实数 α ;

(b) 出口: 并行向量 y , 其中 $y[i] = \alpha \times y[i]$.

```
int
jx_ParVectorScale( double alpha, jx_ParVector *y )
```

- 并行向量的乘积

(a) 入口: 并行向量 x 、 y ;

(b) 出口: 并行向量 z , 其中 $z[i] = x[i] \times y[i]$.

```
int
jx_ParVecMul( jx_ParVector *x,
              jx_ParVector *y,
              jx_ParVector *z )
```

- 并行向量的乘积

(a) 入口: 并行向量 x 、 y 、 z , 实数 α ;

(b) 出口: 并行向量 z , 其中 $z[i] = z[i] + \alpha \times x[i] \times y[i]$.

```
int
jx_ParVecZXY( jx_ParVector *z,
              double        alpha,
              jx_ParVector *x,
              jx_ParVector *y )
```

- 并行向量的内积

- (a) 入口: 并行向量 x 、 y ;
- (b) 出口: 并行向量 x 、 y 的内积.

```
double
jx_ParVectorInnerProd( jx_ParVector *x, jx_ParVector *y )
```

- 并行向量的范数

- (a) 入口: 并行向量 x ;
- (b) 出口: 并行向量 x 的 2-范数.

```
double
jx_ParVectorNorm2( jx_ParVector *x )
```

- 并行向量的 Axy

- (a) 入口: 并行向量 x 、 y , 实数 α ;
- (b) 出口: 并行向量 y ;
- (c) 功能: $y = \alpha \times x + y$.

```
int
jx_ParVectorAxy( double      alpha,
                  jx_ParVector *x,
                  jx_ParVector *y )
```

2. seq_vecop.c

- 串行向量的复制

- (a) 入口: 串行向量 x 、 y ;
- (b) 出口: 串行向量 y .

```
int
jx_SeqVectorCopy( jx_Vector *x, jx_Vector *y )
```

- 串行向量的设定

- (a) 入口: 串行向量 x , 实数 value ;
- (b) 出口: 串行向量 x , 其中 $x[i]=\text{value}$.

```
int
jx_SeqVectorSetConstantValues( jx_Vector *x, double value )
```

- 串行向量的随机设定

(a) 入口: 串行向量 x , 种子 $seed$;

(b) 出口: 串行向量 x .

```
int  
jx_SeqVectorSetRandomValues( jx_Vector *x, int seed )
```

- 串行向量的尺度化

(a) 入口: 串行向量 y , 实数 α ;

(b) 出口: 串行向量 y , 其中 $y[i]=\alpha \times y[i]$.

```
int  
jx_SeqVectorScale( double alpha, jx_Vector *y )
```

- 串行向量的范数

(a) 入口: 串行向量 x ;

(b) 出口: 串行向量 x 的 1-范数.

```
double  
jx_SeqVectorNorm1( jx_Vector *x )
```

- 串行向量的范数

(a) 入口: 串行向量 x 、 y ;

(b) 出口: 串行向量 x 、 y 的逐点 1-范数 $\max\{ |x_i| / |y_i|, i = 0, 1, \dots, n-1 \}$.

```
double  
jx_SeqVectorPointWiseRelNorm1( jx_Vector *x, jx_Vector *y )
```

- 串行向量的乘积

(a) 入口: 串行向量 x 、 y 、 z , 实数 α ;

(b) 出口: 串行向量 z , 其中 $z[i]=z[i]+\alpha \times x[i] \times y[i]$.

```
int  
jx_SeqVecZXY( jx_Vector *z,  
               double      alpha,  
               jx_Vector *x,  
               jx_Vector *y )
```

- 串行向量的乘积

(a) 入口: 串行向量 x 、 y ;

(b) 出口: 串行向量 z , 其中 $z[i]=x[i] \times y[i]$.

```

int
jx_SeqVecMul( jx_Vector *x,
              jx_Vector *y,
              jx_Vector *z )

```

- 串行向量的内积
 - (a) 入口: 串行向量 x 、 y ;
 - (b) 出口: 串行向量 x 、 y 的内积.

```

double
jx_SeqVectorInnerProd( jx_Vector *x, jx_Vector *y )

```

- 串行向量的Axy
 - (a) 入口: 串行向量 x 、 y , 实数 α ;
 - (b) 出口: 串行向量 y ;
 - (c) 功能: $y = \alpha \times x + y$.

```

int
jx_SeqVectorAxy( double alpha, jx_Vector *x, jx_Vector *y )

```

参考文献

- [1] C. S. Feng, S. Shu, J. Xu, C.-S. Zhang. Numerical study of geometric multigrid methods on CPU-GPU heterogeneous computers. *Advances in Applied Mathematics and Mechanics*, 6 (1): 1-23, 2014.
- [2] C. S. Feng, S. Shu, X. Q. Yue. An Improvement to the OpenMP Version of BoomerAMG. *Communications in Computer and Information Science*, 207: 1-11, 2013.
- [3] V. E. Henson, U. M. Yang. BoomerAMG: a parallel algebraic multigrid solver and preconditioner. *Appl. Numer. Math.*, 41: 155-177, 2002.
- [4] Q. K. Liu, Z. Y. Mo, A. Q. Zhang, Z. Yang. JAUMIN: a programming framework for large-scale numerical simulation on unstructured meshes. *CCF Trans. High Perform. Comput.*, 1: 35-48, 2019.
- [5] Z. Y. Mo, X. W. Xu. Relaxed RS0 or CLJP coarsening strategy for parallel AMG methods. *Parallel Computing*, 33 (3): 174-185, 2007.
- [6] Z. Y. Mo, A. Q. Zhang, X. L. Cao, Q. K. Liu, X. W. Xu, H. B. An, W. B. Pei, S. P. Zhu. JASMIN: a parallel software infrastructure for scientific computing. *Front. Comput. Sci. China*, 4 (4): 480-488, 2010.

- [7] J. W. Ruge, K. Stüben. Algebraic multigrid, in Multigrid Methods. Front. Appl. Math., 3: 73-130, 1987.
- [8] Y. Saad. Iterative methods for sparse linear systems (2nd Edition). SIAM, Philadelphia, 2003.
- [9] 舒适. 几类基于几何和分析信息的代数多重网格法及其应用. 博士学位论文, 湘潭: 湘潭大学数学系, 2004.
- [10] S. Shu, D. D. Sun, J. Xu. An algebraic multigrid method for higher-order finite element discretizations. Computing, 77 (4): 347-377, 2006.
- [11] K. Stüben. A review of algebraic multigrid. J. Comput. Appl. Math., 128: 281-309, 2001.
- [12] Y. X. Xiao, S. Shu, P. Zhang, M. Tan. An algebraic multigrid method for isotropic linear elasticity on anisotropic meshes. Int. J. Numer. Meth. Bio., 26 (5): 534-553, 2010.
- [13] 肖映雄, 舒适, 张平文, 莫则尧, 许进超. 求解二维三温能量方程的半粗化代数多重网格法. 数值计算与计算机应用, 4: 293-303, 2003.
- [14] Y. X. Xiao, Z. Y. Zhou, S. Shu. An efficient algebraic multigrid method for quadratic discretizations of linear elasticity problems on some typical anisotropic meshes in three dimensions. Numer. Linear Algebra Appl., 22: 465-482, 2015.
- [15] 徐小文. 并行可扩展代数多重网格算法研究. 博士学位论文, 北京: 中国工程物理研究院研究生部, 2007.
- [16] 徐小文, 莫则尧. 一种新的并行代数多重网格粗化算法. 计算数学, 27 (3): 325-336, 2005.
- [17] X. W. Xu, Z. Y. Mo. Algebraic interface based coarsening AMG preconditioner for multi-scale sparse matrices with applications to radiation hydrodynamics computation. Numer. Linear Algebra Appl., 24: e2078, 2017.
- [18] 徐小文, 莫则尧, 安恒斌. 求解二维三温辐射扩散方程组的一种代数两层网格迭代方法. 计算物理, 26 (1): 1-8, 2009.
- [19] 徐小文, 莫则尧, 安恒斌. 求解大规模稀疏线性代数方程组序列的自适应AMG预条件策略. 中国科学: 信息科学, 46: 1411-1420, 2016.
- [20] X. W. Xu, Z. Y. Mo, X. Q. Yue, H. B. An, S. Shu. α Setup-AMG: an adaptive-setup-based parallel AMG solver for sequence of sparse linear systems. CCF Trans. High Perform. Comput., 2: 98-110, 2020.
- [21] X. Q. Yue, S. Shu, X. W. Xu, Z. Y. Zhou. An adaptive combined preconditioner with applications in radiation diffusion equations. Commun. Comput. Phys., 18: 1313-1335, 2015.

- [22] X. Q. Yue, X. W. Xu, S. Shu. JASMIN-based two-dimensional adaptive combined preconditioner for radiation diffusion equations in inertial fusion research. *East Asian J. Appl. Math.*, 7: 495-507, 2017.
- [23] 周志阳, 徐小文, 舒适, 冯春生, 莫则尧. 二维三温辐射扩散方程两层预条件子的自适应求解. *计算物理*, 29 (4): 475-483, 2012.